Project Checkpoint 5

LCD Peripheral Integration and the Dinosaur Game

CS 342300 Operating Systems — Spring 2026

Name: Christopher

Student ID: 112006233

---

1. Overview

This checkpoint delivers two programs that run on the same multithreading core targeting the 8051 in EdSim51, compiled with SDCC:

- `testlcd` (Part 1) — wires the two-producer / one-consumer structure from the previous checkpoint to real peripherals: the button bank and the numeric keypad act as the two producers, and the LCD module acts as the consumer.
- `dino` (Part 2) — a two-row, cactus-dodging "dinosaur" game built from three threads that read the keypad, run the game logic, and repaint the LCD.

Both programs share the same scheduler in preemptive.c / preemptive.h and the same peripheral drivers (buttonlib, keylib, lcdlib). The keylib driver is used as provided; buttonlib implements the button-priority logic; lcdlib is the provided driver with the first-character bug in LCD_write_string fixed and an LCD_set_symbol helper added for the custom glyphs.

2. Threading model

Every thread runs in register bank 0. Instead of giving each thread its own bank, a context switch pushes the full register image (R0–R7, ACC, B, DPL, DPH, PSW) onto that thread's own stack and pops it back when the thread is resumed. Keeping all threads in one bank means the LCD and keypad library code never has to worry about which register bank an SDCC "ar" temporary lands in.

2.1 Cooperative scheduling

Each per-thread stack is only 16 bytes. A full saved context is 13 bytes, plus a 2-byte return address, which leaves no room to take a switch in the middle of a deep library call (for example LCD_set_symbol → LCD_write_char → delay). The scheduler is therefore run cooperatively: Bootstrap does not arm the Timer 0 interrupt, so the only context switches happen at the explicit ThreadYield() call at the top of each thread's loop. Because a switch is only ever taken at that shallow point — outside any function call or critical section — the saved frame always fits, and the compiler's overlaid locals in helper functions are safe (a helper always runs to completion before its thread yields).

2.2 Scheduler interface

- `ThreadCreate(fp)` — finds a free slot, builds a fake "resumable" stack frame (entry address plus a zeroed register image) so the thread starts at fp, and records its stack pointer.
- `ThreadYield()` — pushes the running thread's context, round-robins to the next live thread, and restores its context.
- `ThreadExit()` — clears the thread's live bit and switches away permanently.

A busy-wait binary semaphore (the SemaphoreCreate / SemaphoreWait / SemaphoreSignal macros carried over from the earlier checkpoints) is used as a one-count lock. Because a thread always releases the lock before it yields, the lock is always free when another thread asks for it, so SemaphoreWait never actually spins — it simply enforces mutual exclusion over the critical region.

Scheduler IRAM usage: `0x38–0x3F` (the saved-SP array, the live-thread bitmask, the current-thread id, and two scratch bytes used while creating a thread).

3. Part 1 — testlcd: peripherals as producers and consumer

3.1 Peripheral drivers

- Button bank — wired to port `P2`. A released switch reads 1, a pressed switch reads 0. `AnyButtonPressed()` returns true when any P2 bit is 0; `ButtonToChar()` returns the ASCII digit of the highest-numbered pressed switch ('7' down to '0'), or the null character when nothing is held — a documented highest-priority policy.
- Keypad — a 4×3 matrix scanned through port `P0`, with the "any key pressed" line on `P3.3` (the AND gate must be enabled in EdSim51). `KeyToChar()` drives one row low at a time and reads the three column inputs to identify the pressed key.
- LCD — an HD44780-style module on port `P1` in 4-bit mode (`DB7–DB4 = P1.7–P1.4`, `RS = P1.3`, `E = P1.2`). `LCD_write_char()` sends a byte as two nibbles, and an lcd_ready flag tells the consumer when the slow module is free.

3.2 Architecture

Three cooperative threads share a 3-slot circular buffer guarded by one binary lock:

- `bank_producer` reads the button bank and enqueues one ASCII byte per press.
- `pad_producer` reads the keypad and enqueues one ASCII byte per press.
- `lcd_consumer` dequeues one byte per pass and writes it to the LCD when the panel is ready.

Each producer is edge-triggered: it remembers whether its device was already held down and enqueues only on the press transition, so holding a key does not flood the buffer. Buffer access is non-blocking — a producer skips this round if the buffer is full and the consumer skips if it is empty — which keeps the cooperative schedule deadlock-free.

3.3 Memory map (testlcd)

Address	Symbol	Meaning
`0x20`	lcd_ready	`LCD busy/ready flag (lcdlib)`
`0x21`	qlock	binary lock protecting the buffer
`0x22`	qcount	items currently in the buffer
`0x23`	qhead	consumer index
`0x24`	qtail	producer index
`0x25-0x27`	ring[3]	the 3-slot circular buffer
`0x38-0x3F`	scheduler	preemptive.c state
`0x40-0x7F`	stacks	16 bytes per thread (T0..T3)

3.4 Screenshot and explanation

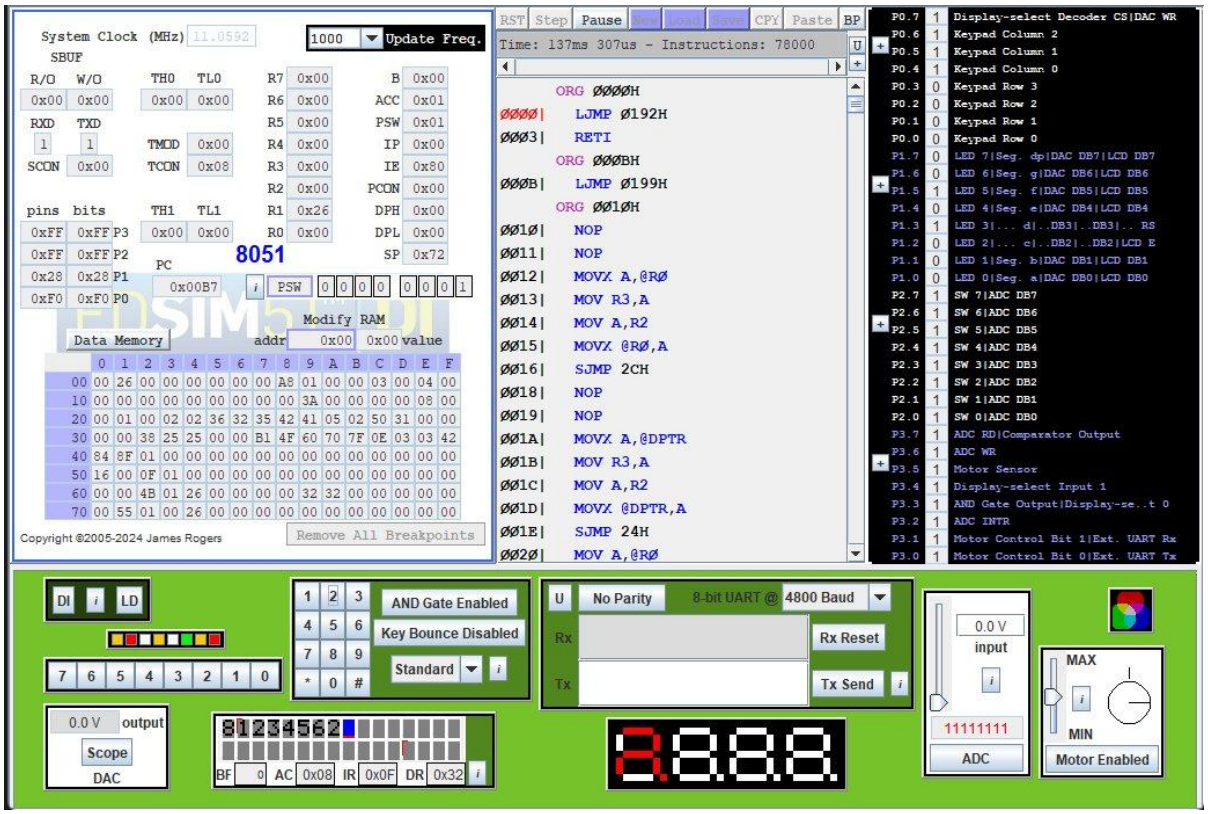

Figure 1 — testlcd.hex: characters entered on the keypad and button bank flow through the bounded buffer and appear on the LCD.

Several keys were pressed; each press was edge-detected, enqueued by a producer, and dequeued by `lcd_consumer`, which wrote it to the LCD. The panel shows the resulting string, the data register `DR` holds the most recent character (`0x32` = '2'), and the address counter `AC` has advanced to the next column with `BF = 0` (not busy). `IE` reads `0x80` — EA on, ET0 off — confirming the cooperative schedule. This demonstrates the complete two-producer / one-consumer pipeline: button bank (P2) and keypad (P0) → ring buffer → LCD (P1).

4. Part 2 — dino: the dinosaur game

4.1 Threads

- `main` (Thread 0) runs the game logic: difficulty selection, cactus motion, collision detection and scoring.
- `pad_thread` (Thread 1) edge-detects keypad presses and posts one key into a single-slot mailbox.
- `draw_thread` (Thread 2) is the only thread that touches the LCD; it repaints the screen from a snapshot of the game state.

Only three threads are created, so the fourth stack slot is never used.

4.2 Gameplay

The dinosaur stays in column 0. The player first types a difficulty digit 0–9 followed by '#'. During play, the [2] key moves the dinosaur to the top row and [8] moves it to the bottom row. Cacti are injected at

column 15 and march one column toward the dinosaur each step; a cactus that reaches the dinosaur's row in column 0 ends the game, while dodging one scores a point. On game over the screen shows "Game Over" and the score.

4.3 Design questions

Data type for the map

There are 16×2 = 32 cells where a cactus could sit, but storing 32 bytes would be wasteful on a part this memory-constrained. Each row is stored as one 16-bit word (`row0` and `row1`), with bit `c` set meaning "a cactus is in column `c`." That is just 4 bytes for the entire field, and shifting the cacti one column toward the dinosaur is a single `>>= 1` per row.

Generating a new cactus

Every `SPAWN_EVERY` steps the game tries to inject a cactus at column 15 (`0x8000`) into a pseudo-randomly chosen row (an 8-bit LFSR picks the row). The injection happens only when column 14 (`0x4000`) is empty in both rows. Forcing that one-column gap guarantees the player can never be trapped: it rules out two cacti in the same column (a flat wall) and two cacti one column apart on different rows (a 45-degree wall).

Difficulty

Difficulty controls the shifting speed. Each step the logic thread waits `(10 - level) × 3 + SPEED_MIN` cooperative yields before moving the cacti, so level 0 is the slowest and level 9 the fastest. (The two pacing constants can be tuned for a comfortable speed.)

4.4 Avoiding race conditions

The variables that could race are the ones shared between threads: the two cactus maps `row0`/`row1`, `dinoRow`, `score`, and the keypad mailbox (`keyBox`/`keyNew`). Under preemption a switch in the middle of an update could let the draw thread paint a half-shifted field (for example a shifted `row0` next to an unshifted `row1`), or let the logic thread read a torn mailbox. Two mechanisms prevent this. First, scheduling is cooperative, so a switch can only happen at a ThreadYield() outside every critical section. Second, every read or update of the shared state is wrapped in the `wlock` lock and kept short, and no thread ever yields while holding it. As a result the draw thread always sees a consistent snapshot and no race occurs.

4.5 Custom glyphs and rendering

Two 8-byte bitmaps (a dinosaur and a cactus) are programmed into the LCD's character-generator RAM at start-up with `LCD_set_symbol`, becoming glyph codes 1 and 2. Each frame the draw thread sets the DD-RAM address to the start of a row and writes 16 cells: the dinosaur glyph in column 0 of its row, a cactus glyph wherever a map bit is set, or a space otherwise.

4.6 Memory discipline (fixed addresses, no C locals)

Because all threads share register bank 0, SDCC places ordinary C locals in low internal RAM. If those locals grow past the register-bank area they can overrun the game state and corrupt it. To make that impossible, every variable in dino.c is given an explicit fixed address and no function uses C locals. This leaves the low IRAM region free for the compiler's own temporaries and guarantees nothing collides with the game state. (This is the same fixed-address discipline used in the earlier checkpoints.)

Address	Symbol(s)	Meaning
`0x20`	lcd_ready	`LCD ready flag (lcdlib)`

Address	Symbol(s)	Meaning
0x21	wlock	lock protecting the shared world
0x22	phase	SETUP / PLAYING / OVER
0x23	level	difficulty 0-9
0x24	dinoRow	dinosaur row, 0 or 1
0x25-0x26	keyBox, keyNew	one-slot key mailbox + flag
0x27-0x28	score	16-bit score
0x29-0x2A	row0	row-0 cactus bitmap
0x2B-0x2C	row1	row-1 cactus bitmap
0x2D-0x30	sbuf[4]	decimal score text
0x31-0x37	thread state	edge/mailbox/logic working bytes
0x38-0x3F	scheduler	preemptive.c state
0x40-0x6F	stacks	three thread stacks
0x70-0x7F	draw/format state	unused 4th slot, reused for state

4.7 Screenshots and explanation

(A) SETUP phase

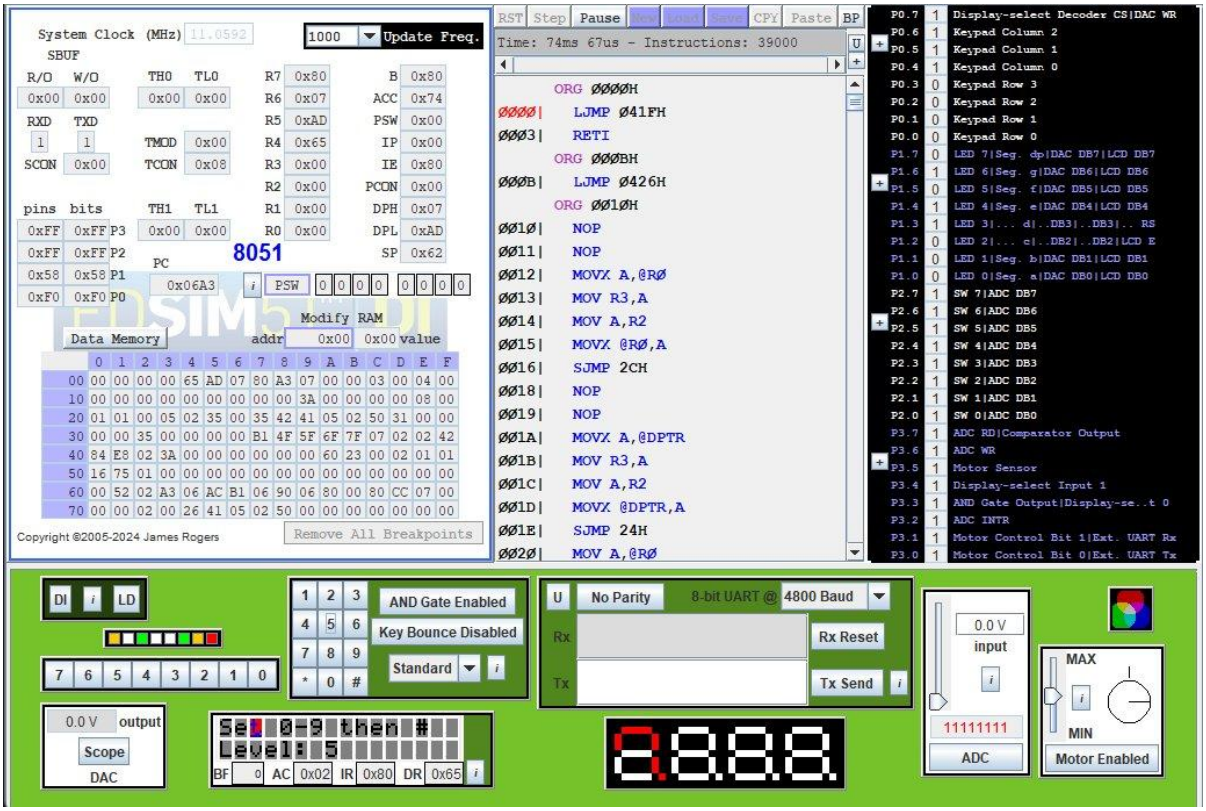

Figure 2 — SETUP: the LCD prompts "Set 0-9 then #" and shows the selected difficulty (Level: 5).

On start, phase = SETUP and the draw thread shows the prompt. Each digit press updates `level` via the mailbox (here the player has chosen 5); the data register reads `DR = 0x65` = '5', the character just written. Pressing '#' ends setup and switches the game to the PLAYING phase.

(B) PLAYING phase

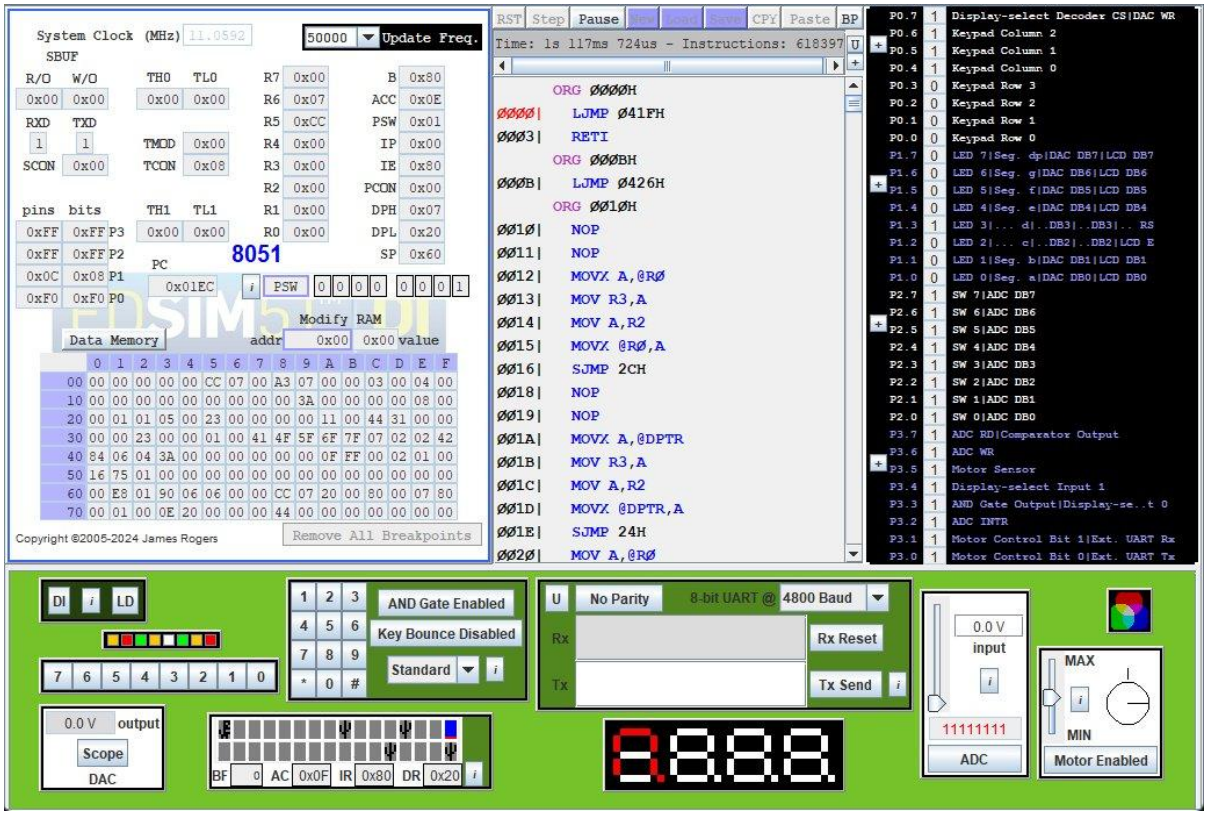

Figure 3 — PLAYING: the dinosaur glyph sits in column 0 while cactus glyphs march in from the right across both rows.

With `phase` = PLAYING the draw thread paints the field each frame from a locked snapshot of `row0` and `row1`. A spawn sets bit 15 (`0x8000`) and every step shifts each row word right by one, so cacti move toward the dinosaur. Pressing [2] or [8] flips `dinoRow` so the dinosaur changes rows to dodge.

(C) GAME OVER phase

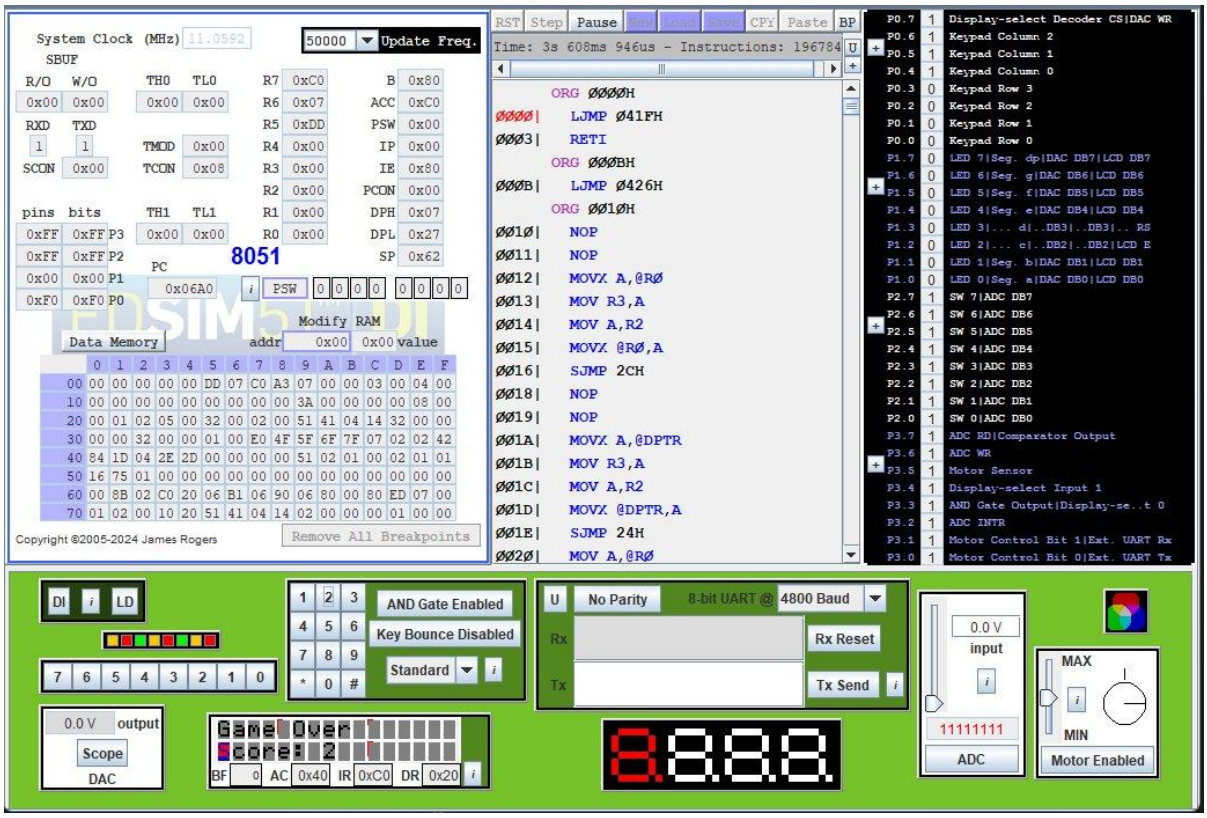

Figure 4 — GAME OVER: after a collision the LCD shows "Game Over" and the final score (Score: 2).

When a cactus reaches the dinosaur's cell the logic thread leaves the play loop, formats `score` into `sbuf` by repeated subtraction (no divide), sets `phase` = OVER, and idles. The draw thread then renders the end screen — the player dodged 2 cacti before colliding.

5. Part 3 — Compilation

Both targets are built from the project directory with make clean followed by make. The Makefile compiles the shared scheduler and drivers and links them into the two hex files.

```
all: testlcd.hex dino.hex

LCD_OBJ  = testlcd.rel preemptive.rel lcdlib.rel buttonlib.rel keylib.rel
DINO_OBJ = dino.rel    preemptive.rel lcdlib.rel keylib.rel
```

[INSERT SCREENSHOT]

Figure 5 — terminal output of make clean and make; testlcd.hex and dino.hex both build with no errors.

Note for the writeup: SDCC prints one harmless warning for `ThreadCreate` — "unreferenced function argument fp" — because fp is consumed inside an `__asm` block that the C front-end cannot see. The generated code is correct.

6. Building and running

- `make clean && make` builds both hex files.

- In EdSim51, enable the keypad AND gate; the button bank, keypad and LCD are on P2, P0 and P1 respectively.
- Load `testlcd.hex` first and verify that a button or keypad press shows the character on the LCD; then load `dino.hex`, choose a difficulty, and play with [2]/[8].